

SENAI "Dr. Celso Charuri" - Unidade Sumaré.

Curso Técnico em Desenvolvimento de Sistemas

Plano de Teste

Sistema Gerenciador de Processos Jurídicos-
JurisVault

1. Introdução e Objetivos	3
2. Histórico de Edição	3
3. Escopo dos Testes	4
4. Estratégia e Técnicas de Teste	4
5. Recursos e Ferramentas	5
6. Gestão de Riscos	5
7. Estimativas e Cronograma	6
8. Comunicação e Encerramento	6

1. Introdução e Objetivos

Apresenta a visão geral do projeto e o que se pretende alcançar com os testes.

- Sistema gerenciador de processos jurídicos "JurisVault".
- Com o avanço da tecnologia, tornou-se essencial utilizar sistemas digitais para organizar e gerenciar informações de forma segura e eficiente. Na área jurídica, advogados e pequenos escritórios precisam controlar processos, prazos e documentos, o que muitas vezes ainda é feito de forma manual ou em planilhas. O objetivo do sistema é facilitar a gestão de processos jurídicos, automatizando tarefas e garantindo maior segurança e organização das informações.
- O software tem como objetivo organizar e armazenar documentos relacionados a processos jurídicos, como petições, provas e contratos, em uma plataforma digital segura. O sistema permitirá o acesso remoto por usuários autorizados, com funcionalidades como controle de versões, autenticação de usuários e rastreamento de atividades. Além disso, utilizará mecanismos de segurança, garantindo a proteção das informações conforme a LGPD, proporcionando mais eficiência e confiabilidade na gestão jurídica.

Entre as principais funcionalidades do JurisVault estão:

- Cadastro de usuários;
- Login e autenticação;
- Controle de acesso por status do usuário;
- Área administrativa;
- Aprovação ou negação de usuários;
- Cadastro e gerenciamento de processos;
- Associação de processos aos usuários;
- Controle de prazos;
- Alertas relacionados aos prazos dos processos;
- Cadastro e gerenciamento de documentos;
- Upload e armazenamento de arquivos;
- Visualização de informações jurídicas;
- Proteção das rotas da aplicação;
- Integração entre Front-end, Back-end e banco de dados;
- O projeto utiliza uma arquitetura composta por Front-end, API Back-end e banco de dados, permitindo a comunicação entre as diferentes partes do sistema;
- O plano de testes será atualizado conforme novas funcionalidades foram implementadas e modificadas durante o desenvolvimento do projeto.

2. Histórico de Edição

Registra as revisões do documento, já que ele deve ser atualizado conforme o projeto evolui.

Mudança	Data	Versão	Responsável
Criação do Plano	10/08/2026	0.0.1	Murilo Demori
Inclusão de registro dos recursos	15/08/2026	0.0.2	Equipe Jurilvault

3. Escopo dos Testes

O que será testado (In-Scope)

- **Autenticação e Controle de Acesso:** Fluxo de login de Administradores e Advogados, incluindo geração e armazenamento do Token JWT no localStorage.
- **Cadastro de Advogados:** Envio de dados pelo formulário React, validação de campos obrigatórios e persistência das informações no banco de dados com status inicial validando.
- **Integração Front-end e Back-end:** Comunicação entre a interface React e os endpoints REST via Axios (POST /auth/login/admin e POST /advogado/cadastro).
- **Tratamento de Erros e Feedback:** Exibição correta de mensagens na interface para credenciais inválidas, falhas de conexão, erro de CORS e campos pendentes.

O que não será testado (Out-of-Scope)

- **Validação Automática do Registro OAB:** Consulta em tempo real com a API externa da OAB para checagem de status profissional.
- **Envio de E-mails Transacionais:** Integração com serviços de e-mail de terceiros (ex: SendGrid, Nodemailer com SMTP externo) para recuperação de senha ou confirmação de cadastro.
- **Testes de Carga e Desempenho:** Simulação de acessos simultâneos em massa ou estresse do banco de dados.
- **Hospedagem e Produção:** Testes em provedores de nuvem (AWS, Vercel, Render), limitando-se ao ambiente local (localhost).

4. Estratégia e Técnicas de Teste

Tipos de Teste

- **Manuais:** Validação funcional das telas em React, focando na usabilidade dos formulários de *Cadastro de Advogados* e *Login Administrativo*, mensagens de feedback visual e navegação após a autenticação.
- **Automatizados:** Testes de API via Postman/Insomnia para validar as rotas do backend (`/advogado/cadastro` e `/auth/login/admin`), verificando o retorno de tokens JWT e os status HTTP (200 OK, 401 Unauthorized, 404 Not Found, etc.), além de testes E2E do fluxo de integração front-to-back.
- **Smoke Test:** Verificação rápida de inicialização executada a cada alteração/deploy, validando se a rota de login do admin e o formulário de cadastro continuam respondendo sem quedas do servidor Node.js ou erros de CORS.

Técnicas de Teste

- **Partição de Equivalência:** Separação de entradas em grupos válidos e inválidos (ex: e-mail/senha cadastrados no banco vs. não cadastrados; campos obrigatórios preenchidos vs. em branco).
- **Análise de Valor Limite:** Validação do comportamento do sistema em cenários de tempo/tamanho, como o limite de expiração do Token JWT (ex: validação após 1 minuto de emissão) e restrições de caracteres nos campos (CPF, registro OAB).
- **Testes Exploratórios:** Testes ad-hoc baseados na experiência para tentar quebrar a aplicação, simular comportamento atípico do usuário (ex: múltiplos cliques no botão de envio), ou testar comportamentos quando o servidor backend é interrompido intencionalmente.

5. Recursos e Ferramentas

PAPEL	NOME	RESPONSABILIDADE
Scrum Master	Murilo Demori Rodrigues	Planejar e acompanhar o cronograma
Desenvolvedor Back-End	Murilo Reis Dota Murilo Demori Rodrigues	Implementar API's em Node.js
Desenvolvedor Front-End	Lívia Carolina Santos Fortunato Kaick Eduardo Araujo Da Silva	Implementar interfaces de interação com o usuário web
Data-base Administrator	Murilo Reis Dota	Implementar, monitorar e garantir a integridade dos dados no banco de dados
Desenvolvedor Mobile	João Rafael Costa Do Nascimento Kaick Eduardo Araujo Da Silva	Implementar interfaces de interação com o usuário de dispositivos móveis
Quality Assurance	Murilo Reis Dota Lívia Carolina Santos Fortunato	Garantir a qualidade do projeto

Ferramentas:

- Postman: realização de testes nas APIs REST;
- Insomnia: realização de requisições e testes das funcionalidades da API;
- MySQL: armazenamento e validação dos dados utilizados durante os testes;
- GitHub: versionamento do código-fonte e acompanhamento das alterações;
- VS Code: ambiente utilizado para desenvolvimento e manutenção do código;
- Render: hospedagem da aplicação e realização de testes no ambiente disponibilizado em nuvem.

Ambiente:

- Ambiente de Desenvolvimento;
- Ambiente de Homologação/Testes;
- Banco de dados MySQL para testes;
- Navegadores modernos para testes de compatibilidade;
- Ambiente hospedado na plataforma Render.

Esses recursos permitem que a equipe da JurisVault execute os testes de forma organizada e integrada ao processo de desenvolvimento, contribuindo para a entrega de uma aplicação mais segura, estável e confiável.

6. Gestão de Riscos

Identifica potenciais problemas técnicos e operacionais do projeto do Sistema Jurídico que podem impactar a qualidade ou o cronograma de testes, acompanhados de suas ações preventivas e corretivas.

- **Risco 1: Falhas de configuração no ambiente de desenvolvimento local (CORS, banco de dados MySQL offline ou divergência de portas entre React e Node.js).**
 - *Mitigação:* Criar um check-list de pré-requisitos do ambiente, padronizar as variáveis no arquivo `.env.example` e executar um *Smoke Test* de conectividade antes de iniciar a suíte de testes completa.
- **Risco 2: Expiração precoce do Token JWT durante a execução dos testes manuais (tempo limite configurado em 1 minuto).**
 - *Mitigação:* Parametrizar o tempo de expiração do JWT via variável de ambiente para ser maior durante a fase de testes (ex: 1 hora) sem alterar o código de produção.
- **Risco 3: Divergência de contratos de dados entre Front-end e Back-end (ex: incompatibilidade de nomes de chaves como login vs email).**
 - *Mitigação:* Validar previamente as rotas no Insomnia/Postman antes de testar a interface e documentar a estrutura esperada do req.body para cada endpoint.
- **Risco 4: Atrasos na integração de telas devido a dependências do backend ou banco de dados.**
 - *Mitigação:* Utilizar dados simulados (*mocks*) no front-end para testar a interface de forma isolada enquanto o endpoint definitivo é finalizado pela equipe de desenvolvimento.

7. Estimativas e Cronograma

ETAPA	DURAÇÃO	PERÍODO
Início da elaboração do DRS, definição do sumário e introdução	6 dias	05/03 11/03
Descrição geral do sistema e levantamento inicial de requisitos funcionais	6 dias	12/03 18/03
Definição de requisitos não funcionais e convenções do documento	6 dias	19/03 25/03
Elaboração das regras de negócio e detalhamento da especificação de requisitos	6 dias	26/03 31/03
Finalização das disposições gerais e revisão geral para entrega do documento	1 dia	01/04 02/04
Configuração do ambiente no VS Code, setup do repositório GitHub e modelagem inicial do banco de dados MySQL.	2 semanas e 5 dias	03/04 24/04
Criação do esquema de banco de dados focado em processos, prazos e usuários, garantindo integridade para transações ACID.	2 semanas e 6 dias	25/04 15/05
Implementação de autenticação com JWT, integração com a API da OAB-SP (RF003) e lógica de proteção de dados (LGPD).	1 mês e 5 dias	16/05 20/06
Desenvolvimento das rotinas de Cadastro de Processos (RF004), Upload de Arquivos (RF002) e Logs de Auditoria (RNF-006).	1 mês e 4 dias	21/06 25/07
Criação das telas de Login, Dashboard de prazos e formulários de processos utilizando	1 mês e 4 dias	26/07 30/08

HTML5, CSS e JavaScript.		
Conectar Front-End ao Back-End; implementar atualização de status (RF005) e exclusão em cascata (RF006).	4 semanas	31/08 27/09
Testes de usabilidade, performance (tempo de resposta < 3s) e compatibilidade cross-browser (RNF-001).	3 semanas e 6 dias	28/09 25/10
Configuração final na plataforma Render e entrega da documentação técnica revisada.	1 semana 4 dias	26/10 06/11

8. Comunicação e Encerramento

Canais de Comunicação

- **Reuniões de Alinhamento e Orientação:** Apresentação periódica dos avanços do projeto e status da suíte de testes para os integrantes do grupo e o orientador do TCC.
- **Gestão e Registro de Inconsistências:** Rastreamento de bugs, tarefas pendentes e correções de código por meio do GitHub Issues/Trello.
- **Comunicação da Equipe:** Alertas e alinhamentos diários de desenvolvimento via Discord, WhatsApp ou Microsoft Teams.

Critérios de Saída (Exit Criteria)

- **Defeitos Críticos Zerados:** Ausência de falhas impeditivas ou erros de nível alto nos fluxos principais (ex: erros 500 no backend, falhas na geração/validação do JWT, bloqueios por CORS ou erro de truncamento de dados no MySQL).
- **Estabilidade dos Fluxos Essenciais:** 100% de aprovação na execução dos testes de *Cadastro de Advogado* (com status inicial validando) e *Login Administrativo*.
- **Validação das Integrações:** Comunicação fluida e sem falhas entre as telas em React, requisições do Axios e o servidor Node.js.

Relatório Final

- **Documentação Técnica do TCC:** Relatório sintético anexado ao documento final do projeto, resumindo a quantidade de testes aplicados, bugs encontrados/corrigidos e taxa de sucesso da aplicação.
- **Evidências de Execução:** Coleta de evidências visuais (telas do React, logs de execução do servidor Node.js e capturas das respostas HTTP no Insomnia/Postman).
- **Parecer Técnico de Qualidade:** Declaração final atestando a estabilidade e prontidão do software para a demonstração prática na banca examinadora.